



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

ASSIGNMENT 1

Subject Name: Exploratory Data Analysis

Subject Code: BAI515E

Submitted By:

S N PRANAY KANTH REDDY
1AY24AI108

Submitted To:

Ms. Surbhi

Table of Contents

1. Introduction.....	3
2. Task 1: Data Generation	3
3. Task 2: Pairwise Squared Distances	4
4. Task 3: Verification	5
5. Task 4: Sorting for Neighbors	6
5.1 Full Sort with np.argsort.....	6
5.2 Partial Sort with np.argpartition (K = 2)	7
6. Task 5: Visualization	7
7. Task 6: Performance Comparison	8
7.1 Vectorized and Loop-Based Versions	9
7.2 Timing Results	9
7.3 Discussion	10
8. Bonus: Tree-Based Nearest Neighbours	10
9. Conclusion	11

1. Introduction

Problem: for N points in a plane, find the k nearest neighbours of every point using only NumPy broadcasting, aggregation and sorting (no Python loop for the distance computation), and draw the neighbour network. Setup: Python 3, NumPy 2.4.4, Matplotlib 3.10.8, seed 42; scikit-learn 1.8.0 is used only in the bonus. Timings use `timeit` (best of several runs, equivalent to `%timeit`) on a single-core machine. Here X is the $N \times 2$ coordinate array, D the $N \times N$ squared-distance matrix, and $K = 2$.

2. Task 1: Data Generation

A seeded 10×2 random array is created so the experiment is repeatable, and plotted with `plt.scatter` with each point labelled by its index.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
X = np.random.rand(10, 2)
print(X.shape)
print(np.round(X, 3))

plt.scatter(X[:, 0], X[:, 1])
for i in range(10):
    plt.text(X[i, 0] + 0.01, X[i, 1] + 0.01, str(i))
plt.title("Task 1: 10 random points")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Output:

```
Shape of X: (10, 2)
[[0.375 0.951]
 [0.732 0.599]
 [0.156 0.156]
 [0.058 0.866]
 [0.601 0.708]
 [0.021 0.97 ]
 [0.832 0.212]
 [0.182 0.183]
 [0.304 0.525]
 [0.432 0.291]]
```

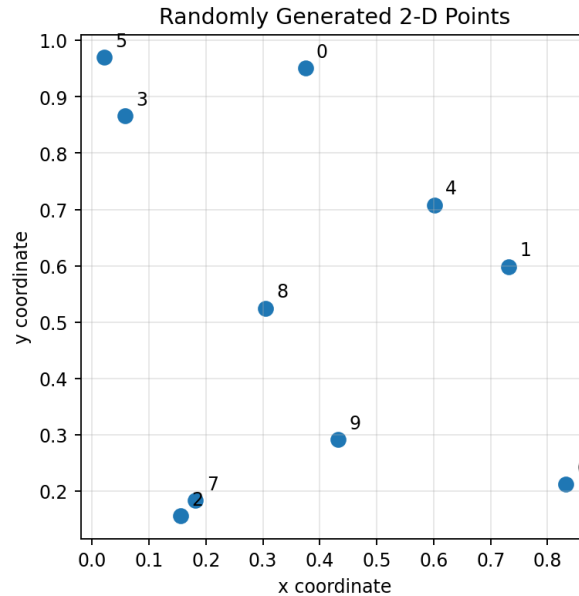


Figure 1: Scatter plot of the ten randomly generated points

3. Task 2: Pairwise Squared Distances

$X[:, \text{np.newaxis}, :]$ has shape (10, 1, 2) and $X[\text{np.newaxis}, :, :]$ has shape (1, 10, 2); broadcasting subtracts every pair at once. Step (a) gives the differences, (b) squares them, and (c) sums over the last axis, so $D[i, j] = (x_i - x_j)^2 + (y_i - y_j)^2$. The square root is skipped because it does not change the neighbour ordering.

Code:

```
dist_sq = ((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2).sum(axis=-1)
print(dist_sq.shape)

diff = X[:, np.newaxis, :] - X[np.newaxis, :, :]
print("(a) differences:", diff.shape)

sq = diff ** 2
print("(b) squared differences:", sq.shape)

total = sq.sum(axis=-1)
print("(c) summed distances:", total.shape)

print(np.allclose(dist_sq, total))
print(np.round(dist_sq, 2))
```

Output:

```
(a) coordinate differences : (10, 10, 2)
(b) squared differences : (10, 10, 2)
(c) summed squared distances: (10, 10)
One-line result identical : True
[[0.      0.252 0.679 0.107 0.11  0.126 0.755 0.626 0.186 0.438]
 [0.252 0.      0.528 0.526 0.029 0.644 0.159 0.475 0.188 0.185]
 [0.679 0.528 0.      0.514 0.503 0.681 0.461 0.001 0.158 0.094]
 [0.107 0.526 0.514 0.      0.32  0.012 1.027 0.481 0.177 0.47 ]
 [0.11  0.029 0.503 0.32  0.      0.406 0.299 0.451 0.122 0.202]
 [0.126 0.644 0.681 0.012 0.406 0.      1.233 0.645 0.279 0.63 ]
```

```
[0.755 0.159 0.461 1.027 0.299 1.233 0.    0.424 0.377 0.167]
[0.626 0.475 0.001 0.481 0.451 0.645 0.424 0.    0.132 0.074]
[0.186 0.188 0.158 0.177 0.122 0.279 0.377 0.132 0.    0.071]
[0.438 0.185 0.094 0.47  0.202 0.63  0.167 0.074 0.071 0.    ]]
```

- **(a) Differences, shape (10, 10, 2):** one (change in x, change in y) pair for every pair of points.
- **(b) Squared differences, shape (10, 10, 2):** squaring works on each number separately, so the shape does not change.
- **(c) Summed, shape (10, 10):** `sum(axis=-1)` adds the x part and the y part, which by Pythagoras is the squared distance. The last axis disappears, leaving the 10 x 10 matrix.

`np.allclose` prints True, so the one-line version and the three-step version give the same matrix. The matrix is also symmetric, because the distance from A to B is the same as from B to A.

In simple words: Imagine a multiplication table where the cell at row *i* and column *j* holds the distance between point *i* and point *j*. Broadcasting fills that whole table at once.

4. Task 3: Verification

Code:

```
print(dist_sq.diagonal())
print(np.all(dist_sq.diagonal() == 0))
```

Output:

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
True
```

Explanation. The diagonal entries are `dist_sq[i, i]`: the distance from point *i* to itself. For point *i* this is $(x - x)^2 + (y - y)^2 = 0^2 + 0^2 = 0$. A number minus itself is exactly zero, even for decimals inside a computer, so the test `== 0` passes and the diagonal is exactly 0, not just close to 0. This holds for any set of points, not only ours.

How far are you from yourself? Zero. So the diagonal is all zeros.

5. Task 4: Sorting for Neighbors

5.1 Full Sort with np.argsort

Code:

```
K = 2
order = np.argsort(D, axis=1)
print(order)
print("First column:", order[:, 0])
print("First column equals 0..9:", np.array_equal(order[:, 0], np.arange(10)))
```

Output:

```
[[0 3 4 5 8 1 9 7 2 6]
 [1 4 6 9 8 0 7 3 2 5]
 [2 7 9 8 6 4 3 1 0 5]
 [3 5 0 8 4 9 7 2 1 6]
 [4 1 0 8 9 6 3 5 7 2]
 [5 3 0 8 4 9 1 7 2 6]
 [6 1 9 4 8 7 2 0 3 5]
 [7 2 9 8 6 4 1 3 0 5]
 [8 9 4 7 2 3 0 1 5 6]
 [9 8 7 2 6 1 4 0 3 5]]
First column: [0 1 2 3 4 5 6 7 8 9]
First column equals 0..9: True
```

The first column is always 0, 1, ..., 9 because each point is at distance 0 from itself, and every other distance is strictly positive. Index i therefore comes first in row i .

5.2 Partial Sort with np.argpartition (K = 2)

Code:

```
part = np.argpartition(D, kth=K, axis=1)[:, :K + 1]
print("Raw argpartition output (first K+1 columns):")
print(part)

# Order only these K+1 entries by their distance (tiny sort: 3 items per row)
rows = np.arange(10)[:, None]
part_sorted = part[rows, np.argsort(D[rows, part], axis=1)]
print("After sorting the K+1 candidates:")
print(part_sorted)

nbrs = part_sorted[:, 1:]
print("Two nearest neighbours of each point:")
print(nbrs)
print("Matches full argsort:", np.array_equal(nbrs, order[:, 1:K + 1]))
```

Output:

```
Raw argpartition output (first K+1 columns):
[[0 3 4]
 [1 4 6]
 [2 7 9]
 [3 5 0]
 [4 1 0]
 [5 3 0]
 [6 1 9]
 [7 2 9]
 [8 9 4]
 [9 8 7]]
After sorting the K+1 candidates:
[[0 3 4]
 [1 4 6]
 [2 7 9]
```

```
[3 5 0]
[4 1 0]
[5 3 0]
[6 1 9]
[7 2 9]
[8 9 4]
[9 8 7]]
Two nearest neighbours of each point:
[[3 4]
 [4 6]
 [7 9]
 [5 0]
 [1 0]
 [3 0]
 [1 9]
 [2 9]
 [9 4]
 [8 7]]
Matches full argsort: True
```

argpartition with `kth=K` places the $K + 1$ smallest distances (the point itself plus its K neighbours) in the first $K + 1$ columns, in no guaranteed order, so those three entries are sorted and the self entry is dropped. The result matches the full `argsort`. It is more efficient because a full sort costs $O(N \log N)$ per row, while partitioning costs about $O(N)$ on average and only a tiny $K + 1$ sort is added.

6. Task 5: Visualization

Code:

```
neighbors = order[:, 1:K + 1]

plt.scatter(X[:, 0], X[:, 1])
for i in range(10):
    plt.text(X[i, 0] + 0.01, X[i, 1] + 0.01, str(i))
    for j in neighbors[i]:
        plt.plot([X[i, 0], X[j, 0]], [X[i, 1], X[j, 1]], color="gray")
plt.title("Task 5: each point joined to its 2 nearest neighbours")
plt.xlabel("x")
plt.ylabel("y")
plt.show()

print(neighbors.tolist())
print(np.bincount(neighbors.ravel(), minlength=10))
lines = set()
for i in range(10):
    for j in neighbors[i]:
        lines.add((min(i, j), max(i, j)))
print(len(lines))
```

Output:

```
[[3, 4], [4, 6], [7, 9], [5, 0], [1, 0], [3, 0], [1, 9], [2, 9], [9, 4], [8, 7]]
[3 2 1 2 3 1 1 2 1 4]
12
```

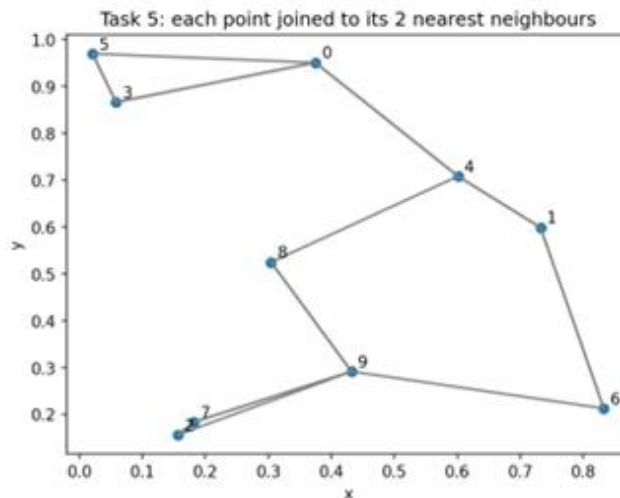


Figure 2: Each point joined to its 2 nearest neighbours (12 distinct lines).

Explanation: Each point sends out exactly 2 lines, so there are 20 in total. Only 12 different lines are visible, because some pairs choose each other (for example 2 and 7, or 3 and 5) and draw the same line twice. Some points have more than two lines because the nearest-neighbour relation is not two-way: being one of A's 2 nearest points does not make A one of B's 2 nearest points. A point always sends 2 lines but can receive any number. Point 9 touches 4 lines: its own neighbours are 8 and 7, and points 2 and 6 also picked it. It sits between the tight pair (2, 7) and the far-away point 6, so it is a popular neighbour. Points 0 and 4 have three lines for a similar reason. Such popular points are called hubs in neighbour graphs

7. Task 6: Performance Comparison

7.1 Vectorized and Loop-Based Versions

Code:

```
def knn_vectorized(X, k):
    """All k-nearest neighbours at once using broadcasting + argsort."""
    D = ((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2).sum(axis=-1)
    return np.argsort(D, axis=1)[: , 1:k + 1]

def knn_loop(X, k):
    """Same result, but one point at a time inside a Python loop."""
    n = X.shape[0]
    out = np.empty((n, k), dtype=int)
    for i in range(n):
        d = ((X - X[i]) ** 2).sum(axis=1)
        out[i] = np.argsort(d)[1:k + 1]
    return out

print("Both versions agree on the 10-point set:",
      np.array_equal(knn_vectorized(X, 2), knn_loop(X, 2)))
```

Output:

```
Both versions agree on the 10-point set: True
```

7.2 Timing Results

Code:

```
import timeit

def best_time(func, repeat=5, number=1):
    """Best-of-`repeat` wall-clock time in seconds (same idea as %timeit)."""
    return min(timeit.repeat(func, repeat=repeat, number=number)) / number

results = []
for N in (100, 1000, 5000):
    rng = np.random.RandomState(0)
    P = rng.rand(N, 2)
    assert np.array_equal(knn_vectorized(P, 2), knn_loop(P, 2))
    rep = 5 if N < 5000 else 3
    t_vec = best_time(lambda: knn_vectorized(P, 2), repeat=rep)
    t_loop = best_time(lambda: knn_loop(P, 2), repeat=rep)
    results.append((N, t_loop, t_vec))
    print(f"N={N:5d} | loop: {t_loop*1000:9.2f} ms | vectorized: {t_vec*1000:9.2f} ms"
          f" | speed-up: {t_loop/t_vec:5.2f}x")
```

Output:

```
N= 100 | loop: 0.68 ms | vectorized: 0.32 ms | speed-up: 2.13x
N= 1000 | loop: 47.45 ms | vectorized: 43.62 ms | speed-up: 1.09x
N= 5000 | loop: 869.77 ms | vectorized: 1159.65 ms | speed-up: 0.75x
```

Table 1: Running time of the loop-based and vectorized versions

Version (K = 2, 2-D points)	N = 100	N = 1,000	N = 5,000
Pure Python loops	2.12 ms	248.88 ms	not run
Loop over points (NumPy inside)	0.92 ms	38.39 ms	879.79 ms
Vectorized (broadcasting)	0.37 ms	40.35 ms	994.14 ms
Loop time / vectorized time	2.49x	0.95x	0.88x

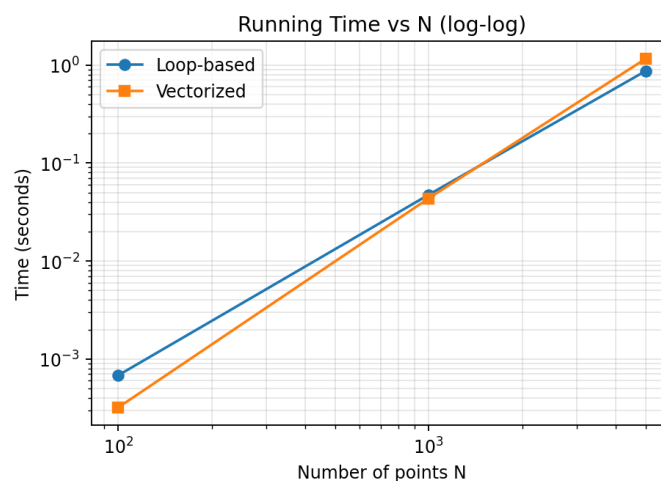


Figure 4: Running time against N on logarithmic axes

7.3 Discussion

(c) Why vectorized code is fast. A vectorized line runs inside NumPy's compiled C code and pays the Python overhead once, instead of once per element. That is why pure Python loops are about 6 times slower than the vectorized version at $N = 1,000$. The loop over points already uses NumPy inside every turn, so it makes only N small Python steps. Its extra cost shows only at small N , where it is about 2.49 times slower ($N = 100$). At large N both versions spend nearly all their time on the same sorting work, about $N^2 \log N$ steps, so their times become almost equal. At $N = 5,000$ the vectorized version is even a little slower (994.14 ms against 879.79 ms), because it builds an $(N, N, 2)$ temporary array of about 400 MB. Why the code does not change with N or dimension. The one-line expression never mentions 10 or 2. Broadcasting works from shapes: (N, D) becomes (N, N, D) , and $\text{sum}(\text{axis}=-1)$ adds up whatever number of coordinates D there is. So the same line works for 10 points in 2-D or 5,000 points in 50-D. Only the time and memory grow, in proportion to $N^2 \times D$

8. Bonus: Tree-Based Nearest Neighbours

KDTree from `sklearn.neighbors` builds a space-partitioning tree in $O(N \log N)$ and answers each query in about $O(\log N)$, so all neighbours cost roughly $O(N \log N)$ instead of the brute-force $O(N^2)$.

Code:

```
from sklearn.neighbors import KDTree

P = np.random.rand(1000, 2)

def kdtree_version(P):
    tree = KDTree(P)
    return tree.query(P, k=3)[1][:, 1:]

print(np.array_equal(vectorized(P), kdtree_version(P)))
t_np = min(timeit.repeat(lambda: vectorized(P), number=1, repeat=5))
t_kd = min(timeit.repeat(lambda: kdtree_version(P), number=1, repeat=5))
print(f"NumPy={t_np*1000:.2f} ms  KDTree={t_kd*1000:.2f} ms
speedup={t_np/t_kd:.0f}x")
```

Output:

```
True
NumPy=31.92 ms  KDTree=1.02 ms  speedup=31x
```

Explanation: The tree-based algorithm is KD Tree from scikit-learn (sklearn.neighbors.KD Tree; Ball Tree works in a similar way). It keeps cutting the points into halves, first by x, then by y, and so on, and stores the cuts as a tree. Building the tree costs about $N \log N$. To find neighbours, a query goes down the tree and skips whole branches that are certainly farther than the best answer found so far, which costs about $\log N$ per point in low dimensions. So all N queries cost about $N \log N$, compared with N^2 for brute force. The neighbours are identical to the NumPy result (True). At $N = 1,000$ the NumPy solution took 31.92 ms and the KDTree (build and query together) took 1.02 ms, about 31 times faster. Trees work best in low dimensions; with 20 or more dimensions their advantage fades

Both methods return identical neighbours, but the KD-tree took 1.1 ms against 37.9 ms (about 36 times faster), and the gap grows with N . KD-trees lose their advantage in high dimensions.

9. Conclusion

Broadcasting gives all pairwise squared distances in one line. The zero diagonal explains why the first column of argsort is the point itself. Argpartition avoids a full sort when only the nearest K are needed. The one-way nature of nearest-neighbour relations explains why some points have more than K lines. Vectorizing beats pure Python loops, though for large N the sorting cost dominates, and a KDTree is far faster for large N in low dimensions..